Complete finite inputs for transitive string C-groups

L047 supplement; inherited roles kept separate

12 October 2026

All six files below are required exact finite premises, not optional validation. The four new complete searches use GAP's finite permutation algorithms and catalogues. The last two are the unchanged, independently reviewed root-authored ten-action boundary. Their requiredness does not promote any finite sample to a uniform proof: the written reduction proves precisely why these complete finite ranges suffice. The full existing Topic B appendix, with its four complete files, remains a separate declared input. Drivers and inherited partial Lean evidence are inventoried separately; no full source formalization is claimed.

Extremal independent strings in S5 and S6

```
# Exhaustive S5/S6 maximal-length independent involutory string markings, modulo
# simultaneous conjugation. A centralizer orbit at each extension preserves
# every already chosen generator, so this prunes no conjugacy class of tuples.
SizeScreen([100000,100000]);;
ReadCase:=function(n)
 local G,inv,target,solutions,nodes,recursor,AllIP,Independent,sol;
 G:=SymmetricGroup(n);target:=n-1;solutions:=[];nodes:=List([0..target],x->0);
 inv:=Filtered(Elements(G),x->Order(x)=2);
 Independent:=function(gs)
  local H,i;
  H:=Subgroup(G,gs);
  return ForAll([1..Length(gs)],i->Subgroup(G,gs{Difference([1..Length(gs)],[i])})<>H);
 end;
 AllIP:=function(gs)
  local H,m,k,v,i;
  H:=List([0..2^Length(gs)-1],m->Subgroup(G,gs{Filtered([1..Length(gs)],i->QuoInt(m,2^(i-1)) mod
  ↪  2=1)}));
  for m in [0..Length(H)-1] do for k in [0..Length(H)-1] do
   v:=Sum(Filtered([0..Length(gs)-1],i->QuoInt(m,2^i) mod 2=1 and QuoInt(k,2^i) mod 2=1),i->2^i);
   if Intersection(H[m+1],H[k+1])<>H[v+1] then return false;fi;
  od;od;return true;
 end;
 recursor:=function(gs,stab,H)
  local cand,reps,x,new,J,types,i;
  nodes[Length(gs)+1]:=nodes[Length(gs)+1]+1;
  if Length(gs)=target then
   if H<>G or not Independent(gs) then return;fi;
   types:=List([1..target-1],i->Order(gs[i]*gs[i+1]));
   if types<>List([1..target-1],i->3) then Error("non-Coxeter maximal marking");fi;
   if ForAny(gs,x->SignPerm(x)<>-1) then Error("nonodd maximal marking");fi;
   if Size(Centralizer(G,Subgroup(G,gs{[2..target]})))<>1 then Error("tail centralizer");fi;
   if Size(Centralizer(G,DerivedSubgroup(Subgroup(G,gs{[2..target]}))))<>1 then Error("derived tail
   ↪  centralizer");fi;
   if not AllIP(gs) then Error("independent maximal string lacks IP");fi;
   Add(solutions,gs);return;
  fi;
  cand:=Filtered(inv,x->not x in H and ForAll(gs{[1..Maximum(0,Length(gs)-1)]},y->x*y=y*x));
  reps:=List(OrbitsDomain(stab,cand,OnPoints),o->o[1]);
  for x in reps do
   new:=Concatenation(gs,[x]);J:=Subgroup(G,new);
   if Independent(new) then recursor(new,Centralizer(stab,x),J);fi;
  od;
 end;
 recursor([],G,TrivialSubgroup(G));
```

```
 Print("N=",n," nodes=",nodes," complete_maximal_markings=",Length(solutions),"\n");
 Print("MARKINGS=",solutions,"\n");
 for sol in solutions do
  Print("EXTREMAL|",n,"|",List(sol,x->List([1..n],i->i^x)),"\n");
 od;
 if Length(solutions)=0 then Error("missing standard marking");fi;
end;;
ReadCase(5);ReadCase(6);
Print("PASS_EXTREMAL_S5_S6\n");QUIT;
```

All small transitive imprimitive actions

```
# Exhaustive degree-six/eight/nine imprimitive high-rank C-markings, modulo
# simultaneous conjugation. A centralizer orbit at each extension preserves
# every already chosen generator, so this prunes no conjugacy class of tuples.
SizeScreen([100000,100000]);;
ReadCase:=function(n,id)
 local G,inv,target,maxrank,solutions,nodes,recursor,AllIP,N,sol;
 G:=TransitiveGroup(n,id);target:=QuoInt(n,2)+1;maxrank:=n-2;solutions:=[];nodes:=List([0..maxrank],
 ↪  x->0);N:=Normalizer(SymmetricGroup(n),G);
 inv:=Filtered(Elements(G),x->Order(x)=2);
 AllIP:=function(gs)
  local H,m,k,v,i;
  H:=List([0..2^Length(gs)-1],m->Subgroup(G,gs{Filtered([1..Length(gs)],i->QuoInt(m,2^(i-1)) mod
  ↪  2=1)}));
  for m in [0..Length(H)-1] do for k in [0..Length(H)-1] do
   v:=Sum(Filtered([0..Length(gs)-1],i->QuoInt(m,2^i) mod 2=1 and QuoInt(k,2^i) mod 2=1),i->2^i);
   if Intersection(H[m+1],H[k+1])<>H[v+1] then return false;fi;
  od;od;return true;
 end;
 recursor:=function(gs,stab,H)
  local cand,reps,x,new,J,types,i;
  nodes[Length(gs)+1]:=nodes[Length(gs)+1]+1;
  if H=G then
   if Length(gs)>=target then
    types:=List([1..Length(gs)-1],i->Order(gs[i]*gs[i+1]));
    Add(solutions,rec(types:=types,gens:=gs));
   fi;return;
  fi;
  if Length(gs)>=maxrank or Index(G,H)<2^Maximum(0,target-Length(gs)) then return;fi;
  cand:=Filtered(inv,x->not x in H and ForAll(gs{[1..Maximum(0,Length(gs)-1)]},y->x*y=y*x));
  reps:=List(OrbitsDomain(stab,cand,OnPoints),o->o[1]);
  for x in reps do
   new:=Concatenation(gs,[x]);J:=Subgroup(G,new);
   if AllIP(new) then recursor(new,Centralizer(stab,x),J);fi;
  od;
 end;
 recursor([],N,TrivialSubgroup(G));
 Print("CASE n=",n," id=",id," order=",Size(G)," structure=",StructureDescription(G),"
 ↪  nodes=",nodes," complete_high_markings=",Length(solutions),"\n");
 Print("MARKINGS=",solutions,"\n");
 for sol in solutions do
  Print("SOLUTION|",n,"|",id,"|",sol.types,"|",List(sol.gens,x->List([1..n],i->i^x)),"\n");
 od;

end;;
for n in [6,8,9] do
 for id in [1..NrTransitiveGroups(n)] do
  G0:=TransitiveGroup(n,id);
  if not IsPrimitive(G0,[1..n]) then ReadCase(n,id);fi;
 od;
od;
Print("PASS_SMALL_TRANSITIVE_IMPRIMITIVE\n");QUIT;
```

Primitive catalogue screening

```
# Complete finite screening for the primitive strict half-degree bound.
# No source-reported C-rank is used. Residual marking searches are separate.
SizeScreen([100000,100000]);;
L047PrimeLength:=n->Length(FactorsInt(n));;
L047E2rank:=function(G)
 local P,cc,ee;
 P:=SylowSubgroup(G,2);cc:=ConjugacyClassesSubgroups(P);
 ee:=Filtered(cc,c->IsElementaryAbelian(Representative(c)));
 return LogInt(Maximum(List(ee,c->Size(Representative(c)))),2);
end;;
L047Screen:=function()
 local n,id,G,ord,r,cl,invgen,cb,disposition,residuals,expected,
 soc,quot,fr,frq,need,A8iso,m24cent,bound;
 residuals:=[];
 expected:=[[6,1],[6,2],[7,5],[8,3],[8,5],[10,4],[10,5],[10,7],[12,2]];
 for n in [2..25] do
  for id in [1..NrPrimitiveGroups(n)] do
   G:=PrimitiveGroup(n,id);ord:=Size(G);r:=QuoInt(n+1,2);
   if ord>=Factorial(n)/2 then continue;fi;
   disposition:="";
   if L047PrimeLength(ord)<r then disposition:="order_chain";
   else
    cl:=Filtered(ConjugacyClasses(G),c->Order(Representative(c))=2);
    invgen:=Subgroup(G,Concatenation(List(cl,Elements)));
    cb:=List(cl,c->1+L047PrimeLength(QuoInt(ord,Size(c))));
    if invgen<>G then disposition:="not_involution_generated";
    elif Maximum(cb)<r then disposition:="involution_centralizer_chain";
    elif n=16 then
     soc:=Socle(G);
     if Size(soc)<>16 or not IsElementaryAbelian(soc) then Error("not affine16");fi;
     # Faithful quotient embeds in GL(4,2)=A8; normal module costs one.
     disposition:="affine16_Whiston_bound7";
    elif n=15 and id=4 then
     if IsomorphismGroups(G,AlternatingGroup(8))=fail then Error("A8 identification");fi;
     disposition:="A8_Whiston_bound6";
    elif n=9 and id=7 then
     soc:=Socle(G);quot:=G/soc;
     if Size(soc)<>9 or not IsElementaryAbelian(soc) then Error("affine9 socle");fi;
     fr:=FrattiniSubgroup(quot);frq:=quot/fr;
     if Size(fr)<>2 or IsomorphismGroups(frq,SymmetricGroup(4))=fail then Error("GL2(3) Frattini
     ↪  quotient");fi;
     disposition:="affine9_normal_module_bound4";
    elif (n=11 and id=6) or (n=12 and id=1) then
     if L047E2rank(G)<>2 then Error("M11 elementary2 rank");fi;
     disposition:="M11_alternate_labels_bound4";
    elif n=8 and id=4 then
     if IsomorphismGroups(G,PrimitiveGroup(7,5))=fail then Error("PSL2(7) repeated action");fi;
     disposition:="same_abstract_group_as_7_5";
    elif n=24 and id=1 then
     m24cent:=List(cl,c->QuoInt(ord,Size(c)));
     if Set(m24cent)<>[7680,21504] or Maximum(m24cent)>=2^20 then Error("M24 tail bound");fi;
     disposition:="M24_Conder_connected_tail_bound";
    else
     Add(residuals,[n,id]);disposition:="complete_marking_census";
    fi;
   fi;
   Print("SCREEN|",n,"|",id,"|",ord,"|",r,"|",disposition,"\n");
  od;
 od;
 if residuals<>expected then Error("unresolved primitive catalogue entry");fi;
 for need in [[10,4],[10,5],[10,7],[12,2],[8,5]] do
  G:=PrimitiveGroup(need[1],need[2]);bound:=L047E2rank(G);
  if need=[8,5] then
   if bound<>2 then Error("PGL2(7) elementary rank");fi;
  elif bound>3 then Error("residual elementary rank");fi;
```

```
 Print("E2RANK|",need[1],"|",need[2],"|",bound,"\n");
 od;
 if IsomorphismGroups(PrimitiveGroup(10,5),SymmetricGroup(6))=fail then Error("degree10 S6");fi;
 # Exact abstract/action identifications used for the residual search limits.
 if IsomorphismGroups(PrimitiveGroup(6,1),AlternatingGroup(5))=fail or
 ↪  IsomorphismGroups(PrimitiveGroup(6,2),SymmetricGroup(5))=fail then Error("degree6 abstract
 ↪  groups");fi;
 if IsomorphismGroups(PrimitiveGroup(7,5),Image(IsomorphismPermGroup(GL(3,2))))=fail then
 ↪  Error("degree7 GL3(2)");fi;
 G:=PrimitiveGroup(8,3);soc:=Socle(G);
 if Size(soc)<>8 or not IsElementaryAbelian(soc) or IsomorphismGroups(G/soc,PrimitiveGroup(7,5))=fail
 ↪  then Error("degree8 affine model");fi;
 # Verify the classical exceptional isomorphism used for affine degree16.
 if IsomorphismGroups(Image(IsomorphismPermGroup(GL(4,2))),AlternatingGroup(8))=fail then
 ↪  Error("GL4(2)=A8");fi;
 for n in [26..63] do
  bound:=n*Product([0..LogInt(n,2)-1],i->n-2^i);
  if bound>=2^(n-1) then Error("Maroti interval bound");fi;
 od;
 Print("PASS_PRIMITIVE_BOUNDS\n");
end;;
L047Screen();QUIT;
```

Complete residual primitive marking searches

```
# Exhaustive connected string C-markings in the bounded primitive residuals.
# Search symmetries only conjugate the entire fixed prefix; every pruning test
# is necessary for an independent string C-marking. Accepted tuples get full IP.
SizeScreen([100000,100000]);;
L047Census:=function(n,id,target,maxrank)
 local G,inv,solutions,nodes,recursor,AllIP,Independent,N,pp,sol;
 if id=0 then G:=AlternatingGroup(n);else G:=PrimitiveGroup(n,id);fi;solutions:=[];nodes:=List([0..m
 ↪  axrank],x->0);N:=Normalizer(SymmetricGroup(n),G);
 inv:=Concatenation(List(Filtered(ConjugacyClasses(G),c->Order(Representative(c))=2),Elements));
 Independent:=function(gs)
  local H,i;
  H:=Subgroup(G,gs);
  return ForAll([1..Length(gs)],i->Subgroup(G,gs{Difference([1..Length(gs)],[i])})<>H);
 end;
 AllIP:=function(gs)
  local H,m,k,v,i;
  H:=List([0..2^Length(gs)-1],m->Subgroup(G,gs{Filtered([1..Length(gs)],i->QuoInt(m,2^(i-1)) mod
  ↪  2=1)}));
  for m in [0..Length(H)-1] do for k in [0..Length(H)-1] do
   v:=Sum(Filtered([0..Length(gs)-1],i->QuoInt(m,2^i) mod 2=1 and QuoInt(k,2^i) mod 2=1),i->2^i);
   if Intersection(H[m+1],H[k+1])<>H[v+1] then return false;fi;
  od;od;return true;
 end;
 recursor:=function(gs,stab,H)
  local cand,reps,x,new,J,types,i,tail,mid,cent,q;
  nodes[Length(gs)+1]:=nodes[Length(gs)+1]+1;
  if H=G then
   if Length(gs)>=target and AllIP(gs) then
    types:=List([1..Length(gs)-1],i->Order(gs[i]*gs[i+1]));
    Add(solutions,rec(types:=types,gens:=gs));
   fi;return;
  fi;
  if Length(gs)>=maxrank or Index(G,H)<2^Maximum(0,target-Length(gs)) then return;fi;
  q:=Maximum(0,target-Length(gs)-1);
  if q>0 and Length(gs)>0 then
   cent:=Centralizer(G,H);
   if Index(cent,Centre(H))<2^q then return;fi;
  fi;
  cand:=Filtered(inv,x->not x in H and ForAll(gs{[1..Maximum(0,Length(gs)-1)]},y->x*y=y*x));
  if Length(gs)>0 then cand:=Filtered(cand,x->x*gs[Length(gs)]<>gs[Length(gs)]*x);fi;
  reps:=List(OrbitsDomain(stab,cand,OnPoints),o->o[1]);
```

```
  for x in reps do
   new:=Concatenation(gs,[x]);J:=Subgroup(G,new);
   if Independent(new) then
    tail:=Subgroup(G,new{[2..Length(new)]});
    mid:=Subgroup(G,gs{[2..Length(gs)]});
    if Intersection(H,tail)=mid then recursor(new,Centralizer(stab,x),J);fi;
   fi;
  od;
 end;
 Print("START n=",n," id=",id," order=",Size(G)," target=",target," max=",maxrank,"
 ↪  involutions=",Length(inv),"\n");
 recursor([],N,TrivialSubgroup(G));
 Print("CASE n=",n," id=",id," order=",Size(G)," nodes=",nodes,"
 ↪  complete_high_markings=",Length(solutions),"\n");
 Print("MARKINGS=",solutions,"\n");
 for sol in solutions do
  Print("SOLUTION|",n,"|",id,"|",sol.types,"|",List(sol.gens,x->List([1..n],i->i^x)),"\n");
 od;
end;;
for l047spec in [[6,1,3,3],[6,2,3,4],[7,5,4,4],[8,3,4,5],[8,5,4,4],[10,4,5,6],[10,5,5,6],[10,7,5,6],
↪  [12,2,6,6],[8,0,6,6],[6,0,4,4]] do
 CallFuncList(L047Census,l047spec);
od;
Print("PASS_PRIMITIVE_MARKINGS\n");QUIT;
```

The inherited finite-neighbor search

```
# Private bounded search. No source theorem or classification is assumed here.
# Enumerate normalized C-markings, characters and the two missing neighbors.
SizeScreen([100000,100000]);;
Independent:=function(ambient,gs)
 local H,i;
 H:=Subgroup(ambient,gs);
 return ForAll([1..Length(gs)],i->Subgroup(ambient,gs{Difference([1..Length(gs)],[i])})<>H);
end;;
AllIP:=function(ambient,gs)
 local masks,groups,a,b,inter;
 masks:=List([0..2^Length(gs)-1],m->Filtered([1..Length(gs)],i->QuoInt(m,2^(i-1)) mod 2=1));
 groups:=List(masks,s->Subgroup(ambient,gs{s}));
 for a in [1..Length(masks)] do for b in [a+1..Length(masks)] do
  inter:=Intersection(masks[a],masks[b]);
  if Length(inter)=Minimum(Length(masks[a]),Length(masks[b])) then continue;fi;
  if Intersection(groups[a],groups[b])<>groups[Position(masks,inter)] then return false;fi;
 od;od;
 return true;
end;;
LiftDiagonal:=function(h,k)
 return PermList(Concatenation(List([1..k],i->i^h),List([1..k],i->k+i^h)));
end;;
RunNormalizedRank:=function(H,k,label,rank)
 local N,inv,nodes,markings,extension_counts,solutions,r,W,Target,Recurse,Extend,output;
 N:=Normalizer(SymmetricGroup(k),H);
 inv:=Filtered(Elements(H),h->Order(h)=2);
 r:=PermList(Concatenation([k+1..2*k],[1..k]));
 W:=Group(Concatenation([r],GeneratorsOfGroup(SymmetricGroup([1..k])),GeneratorsOfGroup(SymmetricGro
 ↪  up([k+1..2*k]))));
 Target:=Group((1,2));
 nodes:=List([0..rank],i->0);markings:=0;extension_counts:=[0,0,0,0,0,0];solutions:=[];
 Extend:=function(gs)
  local bits,hom,C,l,left,right,dist,xc,yc,xs,ys,x,y,i,full,J,stab,orbs;
  for bits in [0..2^rank-1] do
   hom:=GroupHomomorphismByImages(H,Target,gs,List([1..rank],i->(1,2)^(QuoInt(bits,2^(i-1)) mod 2)));
   if hom=fail then continue;fi;
   extension_counts[1]:=extension_counts[1]+1;
   C:=List([1..rank],i->LiftDiagonal(gs[i],k)*r^(QuoInt(bits,2^(i-1)) mod 2));
   for l in [0..rank] do
```

```
    left:=C{[1..l]};right:=C{[l+1..rank]};
    if not ForAll(left,x->ForAll(right,y->x*y=y*x)) then continue;fi;
    dist:=Minimum(l+1,rank+1-l);
    full:=Concatenation(left,[(),r,()],right);
    if ForAny([1..Length(full)],i->1^full[i]>k and Minimum(i-1,Length(full)-i)<dist) then
    ↪  continue;fi;
    extension_counts[2]:=extension_counts[2]+1;
    xc:=Centralizer(W,Group(Concatenation(left{[1..Maximum(0,l-1)]},right,[()])));
    yc:=Centralizer(W,Group(Concatenation(left,right{[Minimum(2,Length(right)+1)..Length(right)]},[(
    ↪  )])));
    xs:=Filtered(Elements(xc),x->Order(x)=2 and x*r<>r*x and (1^x<=k or Minimum(l,rank+2-l)>=dist));
    ys:=Filtered(Elements(yc),y->Order(y)=2 and y*r<>r*y and (1^y<=k or Minimum(l+2,rank-l)>=dist));
    extension_counts[3]:=extension_counts[3]+Length(xs);
    extension_counts[4]:=extension_counts[4]+Length(ys);
    # Quotient x choices by simultaneous conjugations fixing r and all of C.
    stab:=Centralizer(W,Group(Concatenation(C,[r])));
    orbs:=OrbitsDomain(stab,xs,OnPoints);
    for x in List(orbs,o->o[1]) do for y in ys do
     if x*y<>y*x then continue;fi;
     full:=Concatenation(left,[x,r,y],right);
     J:=Group(full);
     if not IsTransitive(J,[1..2*k]) then continue;fi;
     extension_counts[5]:=extension_counts[5]+1;
     if not Independent(W,full) then continue;fi;
     extension_counts[6]:=extension_counts[6]+1;
     if AllIP(W,full) then Add(solutions,rec(C:=gs,character_bits:=bits,split:=l,gens:=full));fi;
    od;od;
   od;
  od;
 end;
 Recurse:=function(gs,stab,J)
  local cand,reps,x,new,K;
  nodes[Length(gs)+1]:=nodes[Length(gs)+1]+1;
  if Length(gs)=rank then
   if J=H and AllIP(H,gs) then markings:=markings+1;Extend(gs);fi;
   return;
  fi;
  if J=H or Index(H,J)<2^(rank-Length(gs)) then return;fi;
  cand:=Filtered(inv,x->not x in J and ForAll(gs{[1..Maximum(0,Length(gs)-1)]},y->x*y=y*x));
  reps:=List(OrbitsDomain(stab,cand,OnPoints),o->o[1]);
  for x in reps do
   new:=Concatenation(gs,[x]);K:=Subgroup(H,new);
   if Independent(H,new) then Recurse(new,Centralizer(stab,x),K);fi;
  od;
 end;
 Recurse([],N,TrivialSubgroup(H));
 output:=rec(label:=label,k:=k,rank:=rank,nodes:=nodes,markings:=markings,extension_counts:=extensio
 ↪  n_counts,solutions:=solutions);
 Print("BOUNDARY_RESULT=",output,"\n");
 if markings=0 then Error("No maximal marking found");fi;
 return output;
end;;
RunNormalized:=function(H,k,label)
 return RunNormalizedRank(H,k,label,k-2);
end;;
RunProduct:=function(a,b)
 local H;
 H:=Group(Concatenation(GeneratorsOfGroup(SymmetricGroup([1..a])),GeneratorsOfGroup(SymmetricGroup([
 ↪  a+1..a+b]))));
 return RunNormalized(H,a+b,Concatenation("S",String(a),"xS",String(b)));
end;;
```

The inherited exact finite action roster

```
Read("search.g");;
results:=[];;
```

```
for ab in [[2,3],[2,4],[3,3],[2,5],[3,4],[2,6],[3,5],[4,4]] do
 Add(results,RunProduct(ab[1],ab[2]));
od;
Add(results,RunNormalized(TransitiveGroup(6,9),6,"T6_9"));;
Add(results,RunNormalized(TransitiveGroup(6,11),6,"T6_11"));;
for result in results do
 if Length(result.solutions)<>0 then Error("surviving boundary marking");fi;
 Print("SUMMARY|",result.label,"|",result.k,"|",result.rank,"|",result.markings,"|",
 JoinStringsWithSeparator(List(result.extension_counts,String),"|"),"|",Length(result.solutions),"\n
 ↪  ");
od;
control:=RunNormalizedRank(Group(()),3,"positive_rank3",0);;
if Length(control.solutions)<>6 then Error("positive control must recover six completions");fi;
if not AllIP(SymmetricGroup(3),[(1,2),(2,3)]) then Error("Coxeter path positive IP control");fi;
if AllIP(SymmetricGroup(3),[(1,2),(2,3),(1,3)]) then Error("redundant family negative IP
↪  control");fi;
if Independent(SymmetricGroup(3),[(1,2),(2,3),(1,3)]) then Error("redundant family negative
↪  independence control");fi;
Print("CONTROL|positive_rank3|",Length(control.solutions),"\n");
Print("PASS_L047_FINITE_BOUNDARY\n");
QUIT;
```